

Code Assessment of the Wrapped M Token v2 Smart Contracts

PUBLIC

Date [June 30, 2026](#)

Produced for



By



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	8
5	System Considerations	9
6	Terminology	12
7	Findings	13
8	Limitations and use of report	21

1 Executive Summary

Dear all,

Thank you for trusting us to help M0 Labs with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Wrapped M Token v2 according to [Scope](#) to support you in forming an opinion on their security risks.

M0 Labs implements the second version of an upgradable, non-rebasing wrapper for the **M** token, supporting yield accrual while respecting the original whitelist of earners and enabling anyone to activate an earner whose address is approved by the separate Registrar contract.

The most critical subjects covered in our audit are asset solvency, functional correctness and precision of arithmetic operations. Security regarding all the aforementioned subjects is satisfactory within the defined trust model.

The general subjects covered are documentation, gas efficiency and the integration of the wrapper into the existing system. Security regarding all the aforementioned subjects is generally good.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Wrapped M Token v2 repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	17 March 2025	5851babb6e72a5f55e5c8869ce5e11c1f39062b1	Initial Version
2	8 April 2025	f79af77c7aa5d3337b2fa3e2d3910c7213824c36	Version with fixes
3	15 June 2026	145a681ea86939bb5d8f4640619767607bb8ca51	Freeze, pause and force transfer features
4	29 June 2026	47165968dec656961e1c0b75e76efe15fb227c94	Fixes for V3
5	30 June 2026	d2374bea6a02cbf9e9bad69a050fbabdbf48290e	Additional fixes

For the Solidity smart contracts, the compiler version `0.8.26` and EVM version `cancun` were chosen.

```
src/
├── EarnerManager.sol
├── ListOfEarningsToMigrate.sol
├── WrappedMTokenMigratorV1.sol
└── WrappedMToken.sol
```

During the review of `Version 3`, the following update in the `common` repository was also reviewed: <https://github.com/m0-platform/common/pull/52>.

After `Version 3`, the scope was updated as follows:

Removed:

```
src/
└── EarnerManager.sol
```

2.1.1 Excluded from scope

Any contracts that are not explicitly listed above are out of the scope of this review. The `WrappedMToken` is an implementation contract intended for use behind the `Proxy`. Direct interaction with the implementation is not a valid use case and is out of scope.

3 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

M0 Labs implements version 2 of a wrapper for the rebalancing **M** token to facilitate interaction with various decentralized finance systems. The wrapper is subject to the original whitelist of eligible earners. The token implementation is deployed behind an upgradable proxy and the governance can override yield recipients for addresses. An additional contract allows admins to make users that are not **M** earners into **wM** earners without the intervention of governance.

The review of version 1 of the wrapper can be found at <https://www.chainsecurity.com/security-audit/m-zero-wrapped-m-token>.

3.1 WrappedMToken

The wrapped **wM** token is ERC-20 compliant with 6 decimals and supports ERC-3009 (transfer of fungible assets via signed authorization, signatures are compliant to ERC-712).

The contract is operational after migration from the v1 contract behind the proxy.

Contrary to v1, earning can be re-enabled after being disabled:

- `enableEarning()` : Checks whether **wM** is in the **MToken** earner list. Registers the contract to start earning in the **M** token, stores the current **M** index in `enableMIndex`.
- `disableEarning()` : Checks whether **wM** has been removed from the **MToken** earner list. Halts earning in the wrapper by resetting `enableMIndex` and stores the last index of the **wM** in `disableIndex`. Important to be executed should the wrapper contract ever be excluded from earning in the **M** token.

M token holders can wrap their tokens using `wrap()` and receive wrapped **M** tokens in exchange. These tokens can be unwrapped using `unwrap()`. Rewards (accrued only if started explicitly) can be claimed directly using `claimFor()`, or automatically when calling `stopEarningFor()`. By default, earning holders receive their realized yield, but holders can specify a recipient for their yield. Governance can override the default recipient of yield for any address, in this case the claimed yield will be transferred to the specified recipient. The yield recipient set by holders has priority over any recipient set by the governance. For an address holding wrapped **wM** tokens to start earning, this must be activated explicitly:

- `startEarningFor()` : Registers the address to start earning. Only possible if earning is enabled by the governance or `EarnManager`. Switches the balance account for this account to the earning mode.
- `stopEarningFor()` : Stops an address from earning. Can be called permissionlessly on the condition that the account has been disabled from earning by the governance or `EarnManager`. The function claims the outstanding yield of this account up until the time of execution, and then converts the balance accounting to non-earning mode. This function is important to be executed when conditions are fulfilled, otherwise the account will continue to earn yield until being stopped.

The yield generated by the non-earners can be claimed and transferred to the `excessDestination` with `claimExcess()`: The wrapper contract earns yield for its full **M** token balance, however since not all wrapped token holders may earn there can be a surplus. This function calculates and transfers the surplus to a predefined address. The current `excessDestination` is set to the distribution vault (<https://etherscan.io/address/0xd7298f620B0F752Cf41BD818a16C756d9dCAA34f>), however, this behavior might change in future versions.

The balances are stored in the same format for earning and non-earning accounts. The balance always represents the amount of `wM` held by an account. For earning accounts, the principal of the account is also stored in order to compute the unrealized yield. The contract keeps track of the amount of earning/non-earning tokens, and of the sum of the rounded principal of earning accounts.

Several view functions exist to inspect the state of the contract. The main view functions:

- `balanceOf()` : Returns the stored balance of an address and does not account for its unrealized yield.
- `totalSupply()` : Returns the sum of `wM` balances for all accounts in earning and non-earning modes. This function does not consider unrealized yield.
- `accruedYieldOf()` : Returns the pending yield of an account. This does not necessarily represent the yield realized by the account after `claimFor()`, as a fee can be taken by an admin or a yield recipient can be set for the account.
- `balanceWithYieldOf()` : Returns the balance of an address and accounts for its unrealized yield. It is to note that this does not necessarily represent the balance of the account after a `claimFor()`, as a fee can be taken by an admin or a yield recipient can be set for the account.

This version of the `wM` token implements exact wrap, unwrap, and transfers. This means that `wrap(X)` will add exactly `X` to the balance, `unwrap(X)` will subtract exactly `X`, and a transfer of `X` will remove exactly `X` from the sender and add exactly `X` to the recipient.

The rounding error introduced by the design choice of operating on exact amounts causes the wrapper to be insolvent, see [projectedEarningSupply\(\) Cannot Cover for Rounding Errors](#). M0 Labs is aware of this issue, but this design choice improves user experience.

3.2 Earner Manager

The `EarnerManager` serves as the source of information when it comes to allowing earning from an address. It is used to allow trusted admins (set by governance) to set some addresses as earning without the need to go through the governance. The order of priority to know whether an address is allowed to earn in `wM` is as follows:

1. Is earner list ignored? if yes return yes, otherwise go to 2.
2. Is the address in the earner list? if yes return yes, otherwise go to 3.
3. Does the address have earner details in the `EarnerManager`? if yes, check that the admin is still a valid admin and return the result, otherwise return no.

If an address has some earner details, the admin can set a fee on the yield generated by the address. The fee can be `0% <= fee <= 100%` of the yield and will be transferred to the admin upon a claim. Admins should call `claimFor` for an account to collect its pending fees before disabling it from earning. If an admin is removed by governance, it immediately loses all pending fees from its managed accounts. Such accounts can be claimed by another admin or they should be stopped from earning by calling `stopEarningFor()`.

The contract `EarnerManager` is deployed behind an upgradable proxy. A trusted account `migrationAdmin` or the governance have the permission to change the implementation of the proxy.

3.3 Migration from v1

The migration from v1 to v2 is responsible for transforming the v1 account model into the v2 account model for the earning addresses, as well as transferring the disabled earning indices if any.

The v1 account model was storing the balance and the last index for the earners, the migration contract transforms the `lastIndex` slot into the `earningPrincipal` by applying the following formula:

$$earningPrincipal = \lfloor \frac{balance}{lastIndex} \rfloor$$

The migration logic is implemented in `WrappedMTokenMigratorV1`. We assume the full list of existing earners on the wrapper is stored when the migrator contract is deployed.

3.4 Changes in Version 2

In **Version 1** of the codebase, the rounding error during token transfers was tracked by a dedicated `roundingError`. This variable was used in the computation of the `excess()` in an attempt to improve the solvency of the contract. The state variable `roundingError` was removed in **Version 2**, hence the rounding error is not tracked anymore. The contract is expected to be insolvent due to roundings, see [Contract can be temporarily insolvent](#).

3.5 Changes in Version 3

- The `EarnManager` has been removed. The only source of truth for earning addresses is now the registrar. As an effect, no fees are taken and the whole yield goes to the configured yield recipient.
- Wrapping and unwrapping have been restricted to be called only from the `SwapFacility`. The `recipient` parameter of `unwrap()` is no longer used. The `M` is sent to the `SwapFacility`, and not to the `recipient`.
- The two `wrapWithPermit()` functions have been removed from the `WrappedMToken`.
- The internal `_claim()` function used when claiming yield was updated to take an extra `skipTransfer` argument. When `skipTransfer` is true, the yield is not transferred to the configured yield recipient.
- The `WrappedMToken` has been updated to implement a role-based access control feature. The `DEFAULT_ADMIN_ROLE` can grant and revoke roles.
- The `WrappedMToken` has been updated to implement a pausability feature. The `PAUSER_ROLE` can pause and unpause the contract. While paused, claiming yield, claiming excess, transfers (ERC20 and ERC3009), wrapping, unwrapping and starting earning for are blocked.
- The `WrappedMToken` has been updated to implement a freeze feature. The `FREEZE_MANAGER_ROLE` can freeze and unfreeze arbitrary addresses. When freezing an account, the account is forced to stop earning. While frozen, the following actions are not allowed for an account: claiming and receiving yield, start earning, stop earning, allowance management (ERC20 and EIP2612), transfers (ERC20 and ERC3009), wrap, unwrap.
- The `WrappedMToken` has been updated to implement a forced transfers feature. The `FORCED_TRANSFER_MANAGER_ROLE` can force the transfer of frozen funds.
- The `WrappedMToken` has been updated to define an `EXCESS_MANAGER_ROLE`. Bearers of this role can update the `excessDestination`. The `excessDestination` was changed from an immutable to a mutable storage variable.
- The `claimExcess()` function now mints `wM` to the `excessDestination` instead of transferring `M` to it.

3.6 Changes in Version 4

- `ListOfEarnersToMigrate` previously accepted the earners array as-is. Its constructor now requires the addresses to be strictly ascending.
- When deauthorizing an earner via `stopEarningFor()`, the yield was previously routed to the configured claim recipient unless the contract was paused. Now the yield is also kept on the account itself when its claim recipient is frozen.

4 Trust Model

4.1 Roles

Roles for each of the contracts in scope are detailed below. For every role, we have detailed the consequences of a role acting maliciously.

The `WrappedMToken` contract is behind a proxy and has the following roles. Capitalized roles may have multiple holders:

- `DEFAULT_ADMIN_ROLE` : can grant and revoke roles. **Fully trusted**.
- `FREEZE_MANAGER_ROLE` : The freeze managers can freeze accounts. They are **partially trusted**: they can temporarily interrupt the normal operation of the protocol, but not misappropriate funds. In case of misuse, an admin can restore normal function.
- `FORCED_TRANSFER_MANAGER_ROLE` : The role grants the power to transfer frozen funds from the original holders to arbitrary recipients. It is **fully trusted**; it can misappropriate frozen funds. If it **colludes** with `FREEZE_MANAGER_ROLE` , it can misappropriate any amount of funds.
- `PAUSER_ROLE` : The role allows pausing and unpausing the token. It can interrupt the normal operation of the protocol, but not misappropriate funds. It is **partially trusted**, as misuse only temporarily interrupts normal operation, and an admin can later restore it.
- `EXCESS_MANAGER_ROLE` : can set the `excessDestination` , i.e. redirect where the surplus yield is sent upon `claimExcess()` . It is **partially trusted**: it cannot misappropriate user funds directly, but can redirect excess yield to an arbitrary address.
- `migrationAdmin` : allows upgrades bypassing the governance. It is **fully trusted** within this implementation, as if this account becomes malicious or gets compromised, it can set arbitrary implementations for the proxy and drain all tokens held by the wrapper.

The users are **untrusted**.

4.2 Other assumptions

Registrar: Source of truth for various information such as the earners list or whether the whitelist is to be ignored and the migrator role. Fully trusted, expected to be the TTG Registrar (<https://etherscan.io/address/0x119FbeeDD4F4f4298Fb59B720d5654442b81ae2c>) contract of the Mzero system.

Mtoken: Rebasing token this wrapper is for. Expected to work according to specifications, including rebasing correctly according to the index report, and valid values returned only. Expected to be the Mzero token (<https://etherscan.io/token/0x866a2bf4e572cbcf37d5071a7a58503bfb36be1b>).

Admins: the admins set in the `em_admins` of the TTG Registrar (<https://etherscan.io/address/0x119FbeeDD4F4f4298Fb59B720d5654442b81ae2c>) are trusted to set the earning statuses and the fee rate properly. They are also trusted not to "steal" yield by marking arbitrary holder addresses as earning and setting a fee rate of 100% without reason. The governance is expected to monitor the actions of the admins and revoke their permissions if necessary. Note: the admins have been removed after **Version 3**.

The `excessDestination` address is not expected to be frozen.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC12 Breaking Changes from V1 to V2

System Consideration	Version 1
----------------------	-----------

The update of the `WrappedMToken` interface introduces some breaking changes for integrators:

- the account model was updated to track the earning principal of the earners instead of the last index. Functions related to `lastIndex` have been replaced by the new `principal` model.
- `wrap()` and `unwrap()` do not return the amount of tokens wrapped or unwrapped anymore.
- unwrapping does not claim the yield anymore. To achieve the same result as in v1 `claimFor()` has to be called before unwrapping.
- the wrapping/unwrapping of the full balance has been removed. It is still possible to achieve this by swapping `M.balanceOf(target)` through the `SwapFacility`, and `claimFor(target)` followed by swapping `wM.balanceOf(target)` through the `SwapFacility`.
- the automatic claim of the yield on transfers has been removed.
- the contract can start earning again after being removed and added again to the earner list.
- some of the getter functions have been renamed (e.g., `vault()` -> `excessDestination()`, `principalOfTotalEarningSupply()` -> `totalEarningPrincipal()`, ...).

SC13 Contract Can Be Temporarily Insolvent

System Consideration	Version 1
----------------------	-----------

Due to the rebasing nature of the `MToken`, the wrapper contract `wM` may be temporarily insolvent. In practice, this should not impact security or UX as the rounding error should balance itself over time with users wrapping and unwrapping, and the yield accumulated by non-earners should be able to cover this small difference. However, it is important to note that this is a potential risk that should be monitored.

SC14 Earner's Yield Cannot Be Claimed When Claim Recipient Is Frozen

System Consideration	Version 3
----------------------	-----------

If an earner's claim recipient is frozen, `claimFor()` reverts and the accrued yield stays on the account. To resolve this, the earner should update their claim recipient via `setClaimRecipient()`, or governance should update the override recipient.

SC15 Effects of Freezing SwapFacility or Portal on WrappedMToken

System Consideration	Version 3
----------------------	-----------

If `SwapFacility` is frozen: `swap()` and `swapOutM()` with `WrappedMToken` as `tokenIn`, and `replaceAssetWithM()` with `WrappedMToken` as `extensionIn` all fail because each path involves a `WrappedMToken` transfer to `SwapFacility`. `Portal` sending `WrappedMToken` directly still works while sending an extension token as source fails because `swapOutM()` is called internally. Receiving with a `WrappedMToken` destination is unaffected as receiving with an extension destination silently falls back to delivering `MToken` due to the low-level call in `_wrap()`.

If `Portal` is frozen: `transferMLikeToken()` fails as `WrappedMToken` transfers to `Portal` are blocked.

SC16 Governance Should Reject New Earners Before Upgrade

System Consideration	Version 1
----------------------	-----------

The upgrade of `wM` from v1 to v2 requires that the list of existing earners is hard-coded into the migrator contract. Therefore, it is critical that no new earners are enabled from the deployment of `WrappedMTokenMigratorV1` until the upgrade of `wM` is finalized.

If the list of accounts in `ListOfEarnersToMigrate` deployed by the migrator contract is incomplete, the upgrade proposal should be rejected by the governance.

By failing to do so, the `lastIndex` will be interpreted as the `earningPrincipal`, breaking the accounting.

SC17 Migrator Prefix Must Not Collide with Other Keys

System Consideration	Version 1
----------------------	-----------

The `WrappedMToken` can be upgraded to a new implementation if there is a value in registrar for a key that matches a hash of a constant prefix and the wrapper address:

```
function _getMigrator() internal view override returns (address migrator_) {
    ...
    uint256(IRegistrarLike(registrar).get(
        keccak256(abi.encode(_MIGRATOR_V1_PREFIX, address(this))))
    ...
}
```

Although unlikely, it is possible that the prefix can be chosen such that it collides with another key in the registrar. For instance, a prefix set to `"wm_claim_override_recipient"` or `"IN_LISTearnings"` would trick the function `_getMigrator()` to read a value from a key that is meant to store another value and not the implementation address.

SC18 Removed Earners Must Be Stopped Earning Yield

System Consideration	Version 1
----------------------	-----------

When governance removes an account from the eligible earning list, the holder of `wM` continues to earn yield until `stopEarningFor()` is called on the wrapper contract for that account. This is similar to the `M` token, where an account removed from the eligible earning list must also be stopped from earning additional yield.

Prior to `Version 3`, this was also valid for admins that were removed from the admin list, that hence cannot make arbitrary accounts earners anymore. When an admin was removed from the admin list, `stopEarningFor()` had to be called for the accounts having the removed admin as an admin.

SC19 Storage Layout of the Proxy

System Consideration	Version 1
The wrapper contract is deployed behind a proxy scheme and implementation upgrades are possible. The storage layout of the new implementation should be carefully checked to be aligned with the existing implementation to ensure compatibility. Note that <code>WrappedMToken</code> inherits multiple contracts that declare their own variables.	

SC20 Yield Can Be Lost

System Consideration	Version 1
----------------------	-----------

- In general, if a `WM` earner cannot either:
- manage the yield they receive, or
 - set a recipient for their yield, or
 - governance did not set a recipient for their yield

then the full yield amount, or part of it, can be lost. This is especially relevant if governance enables the earning globally by setting the flag "earners_list_ignored" in the registrar.

SC21 Yield Is Not Automatically Claimed When the Claim Recipient Is Updated

System Consideration	Version 1
----------------------	-----------

`setClaimRecipient()` does not trigger a claim before updating the recipient. Any yield accrued up to that point will be credited to the new recipient on the next claim.

Users who want the previous recipient to receive accrued yield should manually claim before updating the recipient. Otherwise, any unclaimed yield will be credited to the new address on the next claim.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

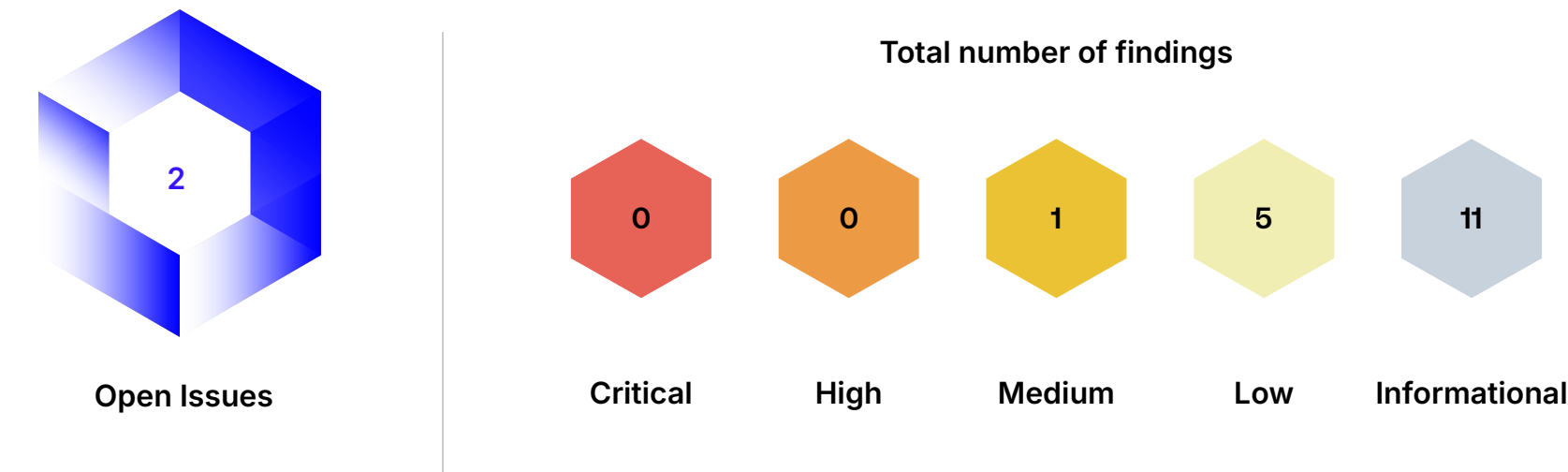
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Medium	#018	Incorrect Tracking of Rounding Error	Specification Changed	✓
Low	#019	Claimed Event Misattributes Yield When skipTransfer Is True	Code Corrected	✓
Low	#020	projectedEarningSupply() Cannot Cover for Rounding Errors	Acknowledged	⊖
Low	#021	Discrepancy of Yield When Earning Is Disabled	Acknowledged	⊖
Low	#022	Earner Can Block stopEarningFor() by Setting a Frozen Claim Recipient	Code Corrected	✓
Low	#023	Unclear Specification for balanceWithYieldOf	Specification Changed	✓
Info	#024	totalEarningPrincipal Can Be Inflated via Repeated Small Transfers	Acknowledged	⊖
Info	#025	Allowance Cannot Be Revoked While Spender Is Frozen	Acknowledged	⊖
Info	#026	Freeze Check on account_ in _wrap Is Easy to Bypass	Acknowledged	⊖
Info	#027	Inaccurate NatSpec	Code Corrected	✓
Info	#028	Missing __init Calls in Initializer	Code Corrected	✓
Info	#029	No Duplicate Address Check in Migrator Earner List	Code Corrected	✓
Info	#030	Possible Gas Optimizations	Code Part. Corrected / Acknowledged	Ⓛ
Info	#031	Remaining TODO Comment	Code Corrected	✓
Info	#032	Rounding Error Is Proportional to the Magnitude of the Index	Acknowledged	⊖
Info	#033	Spurious Event Emissions	Acknowledged	⊖
Info	#034	Wrapper Contract Can Earn WrappedM Yield	Acknowledged	⊖

7.2 Descriptions

#018 Incorrect Tracking of Rounding Error

Correctness	Medium	Version 1	Specification Changed ✓
-------------	--------	-----------	-------------------------

The contract `WrappedMToken` is expected to be insolvent sometimes due to the rounding errors that happen on MToken transfers during wrapping or unwrapping. In v2 of the `WM` implementation, a dedicated state variable `roundingError` is added to track if the sum of all roundings is in favor of the contract or not. A positive value in `roundingError` means the wrapper is insolvent, while a negative value signifies the rounding errors were in favor of the wrapper contract.

The excess amount is reduced if the rounding error is positive (insolvency case):

```
function excess() public view returns (int256 excess_) {
    unchecked {
        ...
        int256 roundingError_ = roundingError > 0 ? roundingError : int256(0);

        // Reduces claimable excess if roundingError is positive, adding an extra layer of
        // safety and solvency.
        return int256(balance_) - int256(earmarked_) - roundingError_;
    }
}
```

In case the `roundingError` is negative, no reduction is applied to the excess amount. Therefore, the surplus due to rounding errors is claimed as excess. However, the `roundingError` is not reset and incorrectly tracks the surplus.

Specification Changed

The tracking logic of the rounding error was completely removed. Nevertheless, the contract can still be insolvent due to rounding errors, but this is a design choice, see [Contract can be temporarily insolvent](#).

#019 Claimed Event Misattributes Yield When `skipTransfer` Is True

Correctness	Low	Version 3	Code Corrected ✓
-------------	-----	-----------	------------------

In `_claim()`, the `Claimed` event is emitted with `claimRecipient_` as the recipient before the `skipTransfer_` check. When `skipTransfer_` is true and `claimRecipient_ != account_`, the yield remains in `account_` but the event incorrectly reports it as sent to `claimRecipient_`.

Code Corrected

`claimRecipient_` is now resolved as `skipTransfer_ ? account_ : claimRecipientFor(account_)` before the event is emitted, so the `Claimed` event always reflects the actual yield recipient.

#020 `projectedEarningSupply()` Cannot Cover for Rounding Errors

Design	Low	Version 1	Acknowledged ☹
--------	-----	-----------	----------------

The function `projectedEarningSupply()` tries to predict how many `M` tokens should be kept for the solvency of the earners but cannot take into account the potential rounding errors that may occur upon

wrapping and unwrapping. This can render the `WrappedMToken` insolvent in some cases. The insolvency can usually be resolved by either waiting for some yield to accumulate, or sending a small amount of `M` to the contract.

Here is an example where the rounding errors can lead to insolvency:

1. Alice is not an earner and wraps `100 x X` tokens at once. This creates a rounding error and the contract is momentarily insolvent.
2. Some time passes and the contract accumulates some yield that covers for the rounding error.
3. `claimExcess()` is called, now the contract should be solvent again.
4. Alice tries to unwrap the amount `X` 100 times; the first 99 times will succeed but the last time will fail because the contract is insolvent again due to rounding error upon transfer of `M` token.

Acknowledged

M0 Labs is aware of this behaviour and accepts the risk of temporary insolvency due to rounding errors.

#021 Discrepancy of Yield When Earning Is Disabled

Correctness	Low	Version 1	Acknowledged 
-------------	-----	-----------	--

Similar to implementation v1, if the governance removes the wrapper contract from the earners list, anyone can trigger a call to stop the wrapper from earning more yield in `M` token in two ways:

1. Call `disableEarning()` in the wrapper contract.
2. Call `stopEarning(wrapperAddr)` in `M` token contract.

The first option is the correct way to stop the wrapper earning more yield and ensure that the accounting in the wrapper is correct. However, since anyone can call `stopEarning()` in the `M` token contract, the second option is also possible. In this case, the accounting of the wrapper will be off depending on the delay with which `disableEarning()` is executed. Theoretically, any delay creates solvency issues for the wrapper as the yield distributed to `wM` holders is larger than the yield earned by the wrapper, hence last users cannot unwrap their tokens.

Acknowledged

M0 Labs is aware of a potential discrepancy in the yield and solvency issues in case of a delay to stop the yield accrual in the wrapper contract.

#022 Earner Can Block `stopEarningFor()` by Setting a Frozen Claim Recipient

Design	Low	Version 3	Code Corrected 
--------	-----	-----------	--

When governance removes an account from the eligible earning list, the account keeps accruing yield until `stopEarningFor()` is called on `WrappedMToken`.

An earner that anticipates being removed from the earner list can therefore call `setClaimRecipient()` with an address that is (or is expected to become) frozen. Every call to `stopEarningFor(account_)` reverts inside `_claim() → _transfer() → _revertIfFrozen(recipient_)`, as long as the account has any non-zero

accrued yield and the contract is not paused. The account keeps its `isEarning` flag and continues to accrue yield indefinitely, defeating the deauthorization mechanism.

Code Corrected

The logic has been updated to skip routing yield to the claim recipient when the recipient is frozen.

#023 Unclear Specification for `balanceWithYieldOf`

Correctness	Low	Version 1	Specification Changed ✓
-------------	-----	-----------	-------------------------

The function `balanceWithYieldOf()` does not consider potential fees paid to admins of managed accounts, or a claim recipient configured for an account. Therefore, the new balance of an account after executing `claimFor()` does not necessarily match with the amount returned by `balanceWithYieldOf()`. The specifications of the function do not describe clearly the return value.

Specification Changed

A note has been added in the function `balanceWithYieldOf()` to clarify that the function does not consider fees paid to admins of managed accounts, or a claim recipient configured for an account.

#024 `totalEarningPrincipal` Can Be Inflated via Repeated Small Transfers

Informational	Version 1	Acknowledged ⊖
---------------	-----------	----------------

Each round-trip transfer of 1 wei between a non-earner and an earner adds `principalUp_` to `totalEarningPrincipal` but only removes `principalDown_`, accumulating a +1 principal error per cycle. Over many iterations this inflates `projectedEarningSupply()` and reduces `excess()`. The attack is economically irrational as the gas cost per round-trip far exceeds any benefit.

Acknowledged

M0 Labs acknowledged this behavior and stated:

The asymmetric rounding is intentional: `totalEarningPrincipal` is incremented with the rounded-up principal (`_addTotalEarningSupply`) and decremented with the rounded-down principal (`_subtractTotalEarningSupply`), deliberately overestimating the principal to add a safety margin to `excess()` calculations. The resulting +1 per-cycle accumulation only inflates `projectedEarningSupply()` and reduces claimable `excess()`, i.e. it errors on the side of solvency and never against users.

#025 Allowance Cannot Be Revoked While Spender Is Frozen

Informational	Version 3	Acknowledged ⊖
---------------	-----------	----------------

The overridden `_approve()` reverts when either the owner or the `spender_` is frozen. As a result, the owner cannot reduce or zero out an allowance granted to a spender that is frozen.

The impact is limited: a frozen spender cannot pull funds either, as `transferFrom()` reverts in `_transfer()` on the frozen spender. The residual concern is a race condition. If the spender is later unfrozen, the stale allowance becomes live again and only then can the owner revoke it.

Acknowledged

M0 Labs is aware of this behavior and stated:

We keep the behavior as is to maintain a consistent freeze model that prevents users from interacting with a frozen account. The impact is limited, as a frozen spender cannot pull funds either (`transferFrom()` reverts in `_transfer()` on the frozen spender), and a frozen account is unlikely to be unfrozen in practice.

#026 Freeze Check on `account_` in `_wrap` Is Easy to Bypass

Informational	Version 3	Acknowledged
---------------	-----------	--------------

`_revertIfFrozen($, account_)` prevents a frozen account from wrapping M, but the check is easy to bypass: a frozen account can transfer their M tokens to an unfrozen address they control, which can then wrap freely.

Acknowledged

M0 Labs acknowledged this behavior and stated:

We retain the `account_` freeze check for consistency with the freeze model applied uniformly across all `wM` entry points (wrap, unwrap, transfer, approve). The freeze applies only at the `wM` level; it cannot constrain what a holder frozen on `wM` does with their underlying `M`, which lives in a separate token contract outside `wM`'s control. Such a holder can always move their `M` to another address they control and wrap from there, so this bypass is inherent and not something `wM` can guard against.

#027 Inaccurate NatSpec

Informational	Version 1	Code Corrected
---------------	-----------	----------------

Several NatSpec comments are inaccurate.

- `unwrap()`: the `@notice` states the M is sent to `recipient` and the `@param recipient` describes it as "The account receiving the withdrawn M", but `recipient` is ignored and M is always sent to the SwapFacility contract.
- `enableEarning()` and `disableEarning()`: both state "if it has never been done", but earning can be disabled and re-enabled multiple times.

Code Corrected

All three NatSpec comments have been updated: `unwrap()` now documents that M is sent to the SwapFacility and that `recipient` is unused; `enableEarning()` and `disableEarning()` now read "not already enabled/disabled" respectively.

#028 Missing `__init` Calls in Initializer

Informational

Version 3

Code Corrected 

The `initialize()` function does not call `__AccessControl_init()`, `__ERC165_init()`, or `__Context_init()`, despite `WrappedMToken` inheriting from `AccessControlUpgradeable` (via `Freezable`, `Pausable`, and `ForcedTransferable`). While these functions are currently no-ops in the used OpenZeppelin version, the omission is a maintenance risk: if a future version introduces meaningful initialization logic, the missing calls would silently produce uninitialized state.

Code Corrected

`initialize()` has been updated to call `__Context_init()`, `__ERC165_init()`, and `__AccessControl_init()`.

#029 No Duplicate Address Check in Migrator Earner List

Informational

Version 1

Code Corrected 

The constructor of `WrappedMTokenMigratorV1` does not check for duplicate addresses in the `earners_` array. If an address appears twice, the second pass reads the already-computed `earningPrincipal` from the first pass as if it were still a `lastIndex`, yielding a corrupted principal approximately equal to the index value itself (around 10^{12}). For most earners holding under 1 million tokens (correct principal well below 10^{12}), this massively inflates their principal and `projectedEarningSupply()`. For large holders above that threshold (correct principal above 10^{12}), it severely understates their principal and their accrued yield.

Governance is expected to verify the earner list before setting the migrator, but an on-chain duplicate check would enforce this invariant without relying on an off-chain process.

Code Corrected

The constructor of `ListOfEarnersToMigrate` now requires the earners array to be strictly ascending by address, enforcing unique addresses in the `earners_` array.

#030 Possible Gas Optimizations

Informational

Version 1

Code Partially Corrected / Acknowledged 

We provide a non-exhaustive list of optimizations to make the codebase more efficient in terms of gas:

1. The function `getEarnerDetails()` performs redundant SLOADs when reading from the storage pointer `details_`. Loading the slot into memory once would be more efficient in terms of gas costs.
2. Functions `startEarningFor(address[])` and `stopEarningFor(address[])` iterate through all input accounts and perform multiple external calls to `EarnerManager` to get earning details for each account individually. However, a single call to function `EarnerManager.getEarnerDetails(address[])` can be used instead.
3. Reading or writing the flag `hasClaimRecipient` in struct `Account` adds one SLOAD or SSTORE operation, which can be avoided if reading or writing to mapping `_claimRecipients` directly.
4. The sanity checks in function `_mint()` can be performed earlier to save gas in case of invalid parameters.

Code Partially Corrected

- 1. FIXED: The EarnerDetails is now loaded in memory.
- 2. FIXED: The EarnerManager has been removed in Version 3.

Acknowledged

M0 Labs acknowledged points 3 and 4.

#031 Remaining TODO Comment

Informational	Version 1	Code Corrected
---------------	-----------	----------------

In EarnerManager a TODO comment is present in function isAdmin.

Code Corrected

The TODO comment has been removed from the codebase.

#032 Rounding Error Is Proportional to the Magnitude of the Index

Informational	Version 1	Acknowledged
---------------	-----------	--------------

Since the principal amounts are computed as $\frac{presentAmount}{index}$ with limited precision, the rounding errors are proportional to the magnitude of the index. As for the MToken, the governance should evaluate the impact of the rates and rate models they enable for the protocol in order to avoid the index, and thus the rounding errors, growing too large too fast.

Acknowledged

M0 Labs is aware of the behavior described above.

#033 Spurious Event Emissions

Informational	Version 3	Acknowledged
---------------	-----------	--------------

The following occurrences were identified:

- 1. _setExcessDestination() emits ExcessDestinationSet even when excessDestination_ is equal to the already stored excessDestination.
- 2. setClaimRecipient() emits ClaimRecipientSet even when claimRecipient_ is equal to the recipient already set for msg.sender. In addition, both storage slots (_claimRecipients[msg.sender] and hasClaimRecipient) are written unconditionally, incurring redundant SSTORE operations on a no-op call.

Acknowledged

M0 Labs acknowledged the behavior and stated that they prefer the simpler unconditional implementation.

#034 Wrapper Contract Can Earn WrappedM Yield

Informational Version 1 Acknowledged 

One can donate `wM` to the wrapper contract and trigger the start of earnings, allowing yield to accumulate over time. Currently, the wrapper contract does not rely on its own balance, so this is not problematic. Governance could redirect the yield to an arbitrary address by overriding the recipient for the wrapper's yield. Since the current implementation of the wrapper contract lacks functionality to transfer out such tokens, they would be locked.

Acknowledged

M0 Labs is aware of the behaviour described above.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.